# Week 6 Pandas Notes - Page 1 of 2

## 1. Why Pandas matters

Pandas is a Python library for handling structured data. In digital forensics and cyber security, it helps analyse logs, login attempts, device inventories, and incident records quickly and repeatably.

## 2. Series versus DataFrame

A **Series** is a one-dimensional labelled column. A **DataFrame** is a two-dimensional table with named columns and row indexes. Most coursework operations use a DataFrame loaded from CSV.

```
import pandas as pd
logs = pd.read_csv("login_attempts.csv")
print(logs.head())
print(logs.shape)
print(logs.dtypes)
```

## 3. Inspect before analysis

Use **head()** to preview rows, **info()** to identify missing values and data types, **describe()** for numeric statistics, and **columns** to verify field names. This prevents KeyError and incorrect comparisons.

## 4. Select rows and columns

Square brackets select columns; **loc** uses row/column labels and **iloc** uses integer positions. A boolean mask selects rows matching a condition.

```
cpu = logs["cpu"] if "cpu" in logs.columns else None
# Example for a server inventory DataFrame:
# servers.loc[0:3, ["host", "status"]]
# servers.iloc[:4, :2]
```

## 5. Filtering logic

Use comparison operators such as ==, !=, >, <, >= and <=. Combine Pandas conditions with **&** (AND) or **|** (OR), wrapping each condition in parentheses. Filtering does not alter the original table unless reassigned.

```
at_risk = servers[(servers["cpu"] > 80) &
                  (servers["status"] != "Offline")]
```

## Key takeaway

A reliable workflow starts with importing the CSV, checking its schema, selecting the needed fields, and only then applying filtering or calculations.

# Week 6 Pandas Notes - Page 2 of 2

## 6. Sorting, grouping and aggregation

**sort_values()** ranks records by a field. **groupby()** partitions rows by category. Aggregations such as count, size, mean, min, max and sum produce useful summaries, for example average CPU load by region or failed logins by IP address.

```
by_region = servers.groupby("region")["cpu"].mean()
counts = servers["status"].value_counts()
ranked = servers.sort_values("cpu", ascending=False)
```

## 7. Apply and lambda

**apply()** runs a function across values; **lambda** defines a short inline function. They are useful for adding risk labels or normalising fields. Prefer vectorised comparisons for simple operations on large datasets.

```
servers["risk"] = servers["cpu"].apply(
    lambda value: "High" if value >= 80 else "Normal")
```

## 8. Cleaning and error handling

Missing values and inconsistent types can distort results. Use **isna().sum()** to count gaps, **to_numeric(errors="coerce")** to convert numeric text, and **dropna()** or **fillna()** when justified. Avoid silently deleting evidence; record cleaning decisions.

## 9. Reporting and charts

Use **to_csv(index=False)** to export a reproducible result. Bar charts are suitable for categorical counts such as statuses, regions or suspicious IPs. Label axes, provide a clear title and save the chart as PNG.

```
summary = servers.groupby("status").size()
summary.plot(kind="bar", title="Servers by status")
plt.ylabel("Number of servers")
plt.tight_layout()
plt.savefig("servers_by_status.png")
```

## 10. Cyber security example

For login records: import the file; verify IP, outcome and timestamp fields; filter failed attempts; group by source IP; count failures; sort descending; flag IPs exceeding a documented threshold; export a CSV and a chart. A high count is an investigative lead, not proof of an attack.

## Debugging checklist

**KeyError:** wrong column name. **ValueError:** invalid conversion or shape. **TypeError:** incompatible types. Check the traceback, inspect dtypes, print intermediate results and rerun after fixing.

## Revision summary

Remember the sequence: **load** → **inspect** → **select/filter** → **group/sort** → **transform** → **visualise/export**. Keep original evidence intact and document any assumptions.